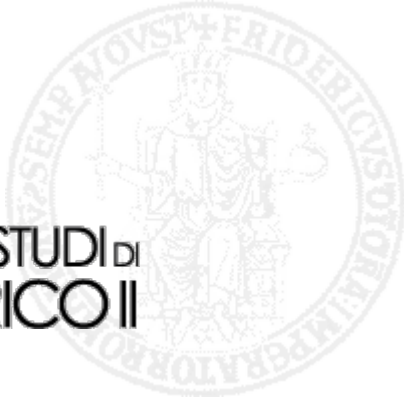




UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Cloud Platform And Infrastructure as Code

*Scalamarket: Using API, Kubernetes
and AWS for developing a
Microservice-based E-Commerce
Platform*

De Lucia Simone M63001720

Indice

1	Presentazione del Progetto	1
1.1	Descrizione Generale	1
1.2	Requisiti dell'Applicativo	1
1.3	Il Sito Web implementato con Flask	3
2	Orchestrazione Kubernetes	5
2.1	Infrastruttura Barebone	5
2.2	Infrastruttura con Ridondanza e Ingress	6
2.3	Guida all'Avvio	7
3	Deployment in AWS	9
3.1	Deployment tramite Infrastructure-as-Code (Terraform)	9
3.2	Deployment tramite AWS Management Console (GUI)	10
4	Proposte di Integrazione e Sviluppo Futuro	11
4.1	Panoramiche sull'utilizzo in locale (On-Premise / Edge Computing)	11
4.2	Benchmark Prestazionali (Richieste/sec)	12
5	Documentazione Endpoints API	14



Capitolo 1

Presentazione del Progetto

1.1 Descrizione Generale

Scalamarket è una moderna piattaforma e-commerce basata su un'architettura a microservizi, progettata per garantire elevata scalabilità, affidabilità e flessibilità. Il sistema gestisce l'intero ciclo di vita dell'acquisto online: dalla registrazione degli utenti, alla consultazione del catalogo, fino all'elaborazione degli ordini e alla gestione logistica delle spedizioni. L'intera infrastruttura è stata ideata per essere distribuita in modo efficiente tramite Kubernetes ed eventualmente ospitata in ambiente cloud AWS, fornendo un backend solido, scalabile e supportato da API RESTful.

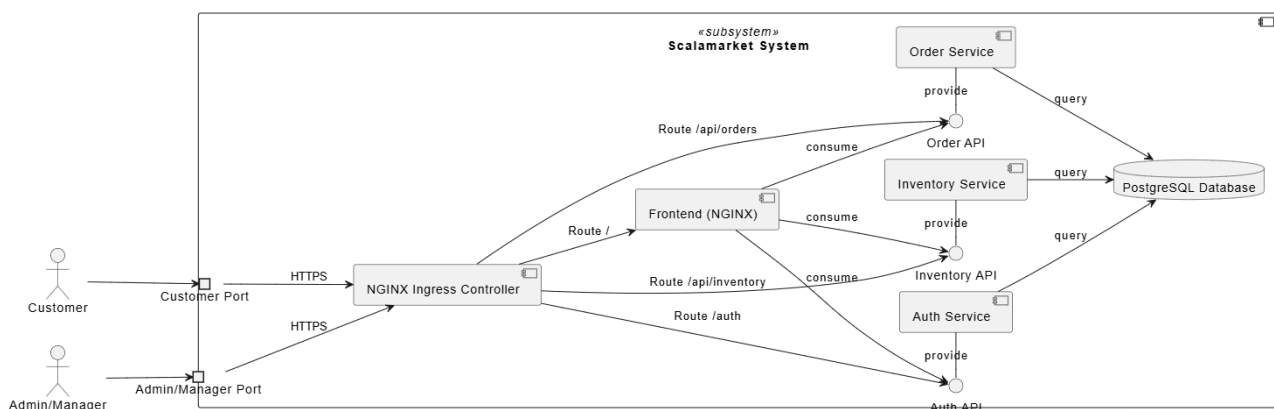


Figura 1.1: Diagramma dei Componenti UML

1.2 Requisiti dell'Applicativo

L'applicativo è stato sviluppato per soddisfare una serie di requisiti architetturali, funzionali e di business specifici, documentati nei requisiti di progetto:

- **Autenticazione e Autorizzazione (Auth Service):** Il sistema deve permettere la registrazione e il login sicuri sia per i clienti



(Client) che per gli amministratori (Admin). L'amministratore deve avere i privilegi per gestire le utenze.

- **Gestione Inventario (Inventory Service):** Il sistema deve gestire un catalogo dinamico di articoli, tenendo traccia delle quantità ripartite in molteplici filiali. Deve inoltre impedire la creazione di ordini per articoli non disponibili.
- **Gestione Ordini e Logistica (Order Service):** L'applicativo deve processare il carrello dell'utente, allocando in tempo reale i prodotti necessari e scalandoli in modo intelligente dalle filiali che ne hanno disponibilità. Deve inoltre tracciare l'occupazione dei mezzi logistici, gestendo i furgoni in uso e quelli a disposizione.
- **Architettura a Microservizi e Dati Isolati:** Per massimizzare la modularità, ogni servizio deve possedere una propria logica di business e interfacciarsi con il proprio database PostgreSQL indipendente, disaccoppiando così i domini applicativi.

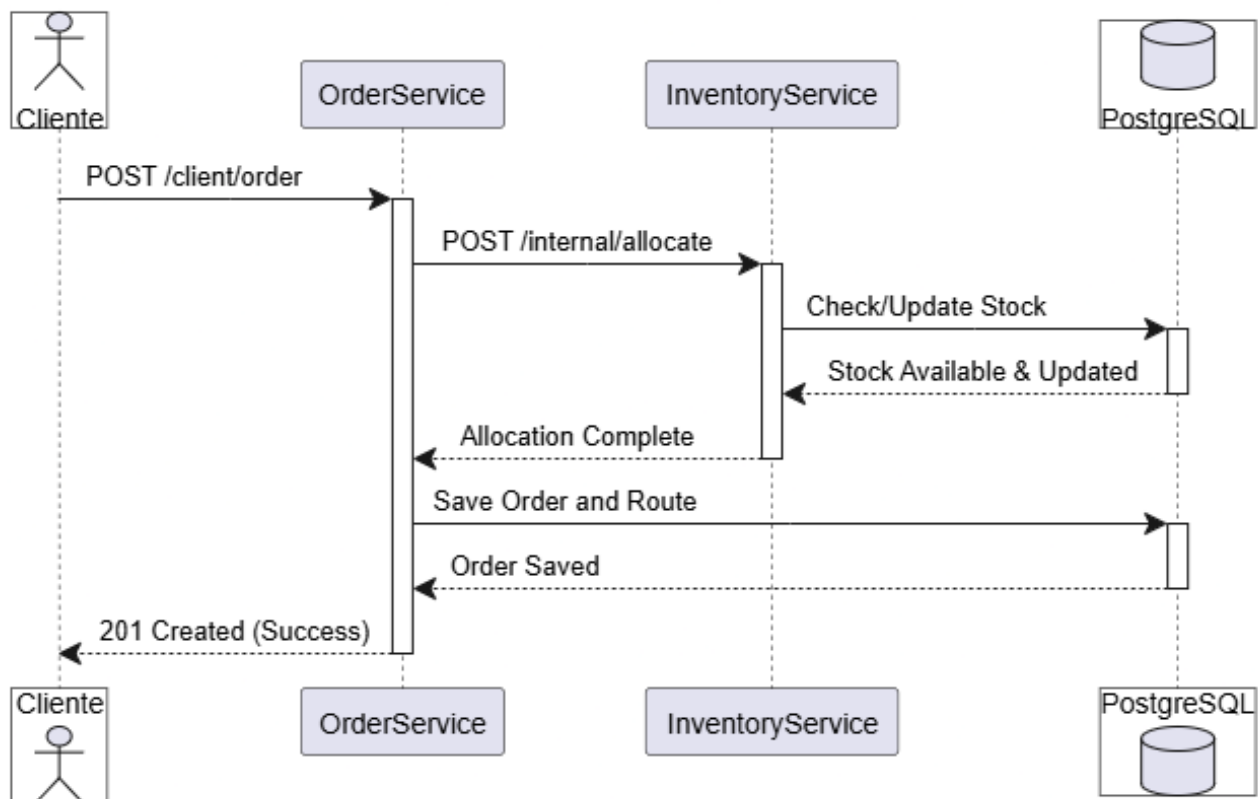


Figura 1.2: Diagramma di Sequenza



1.3 Il Sito Web implementato con Flask

Il lato client e il pannello di gestione amministrativa sono stati realizzati mediante un'interfaccia web responsive integrata con il backend sviluppato in Flask. L'applicazione Flask espone e gestisce tutti gli endpoint necessari, mentre la UI comunica costantemente con questi servizi tramite chiamate asincrone.

- **Interfaccia Cliente:** L'utente può scorrere l'intero catalogo, visualizzare i prezzi e le disponibilità massime, aggiungere elementi al carrello e, una volta inviato l'ordine, monitorarne lo stato e i dettagli dei prodotti tramite un'apposita cronologia.
- **Interfaccia Amministratore:** L'admin ha accesso a una dashboard completa che gli permette di gestire l'inventario (con possibilità di ricerca rapida tramite nome e identificativo), controllare in tempo reale lo stato logistico (veicoli in uso, totali e disponibili), visionare lo storico ordini globale e monitorare/eliminare gli account utenti.

TODO: Inserire qui degli screenshot rappresentativi delle schermate del sito web (es. Dashboard Utente, Pannello di Controllo Admin).

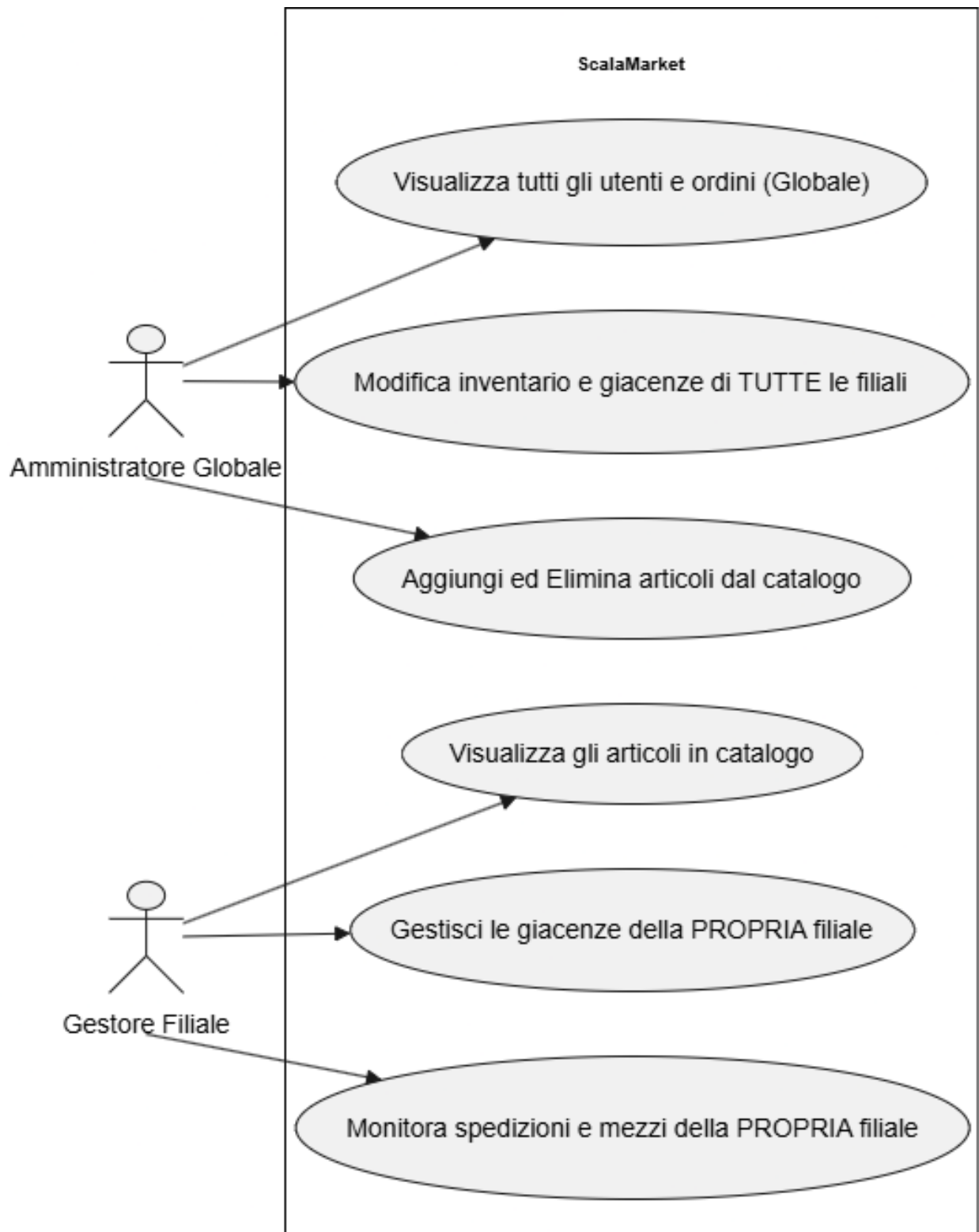


Figura 1.3: Diagramma dei Casi d'Uso



Capitolo 2

Orchestrazione Kubernetes

2.1 Infrastruttura Barebone

La prima configurazione Kubernetes realizzata è definita **Barebone**: è operativa e funzionante, ma priva di qualsiasi meccanismo di ridondanza. Ogni componente dell'applicazione viene eseguito come un singolo Pod, il che configura un *Single Point of Failure* per ogni livello dello stack.

I componenti dell'infrastruttura Barebone sono i seguenti:

- **frontend** (nginx): Deployment con 1 replica, esposto all'esterno tramite un Service di tipo LoadBalancer sulla porta 8080.
- **auth-service** (Flask): Deployment con 1 replica, raggiungibile internamente tramite Service ClusterIP sulla porta 5001.
- **inventory-service** (Flask): Deployment con 1 replica, raggiungibile internamente tramite Service ClusterIP sulla porta 5002.
- **order-service** (Flask): Deployment con 1 replica, raggiungibile internamente tramite Service ClusterIP sulla porta 5003.
- **postgres**: Deployment con 1 replica, con storage persistente garantito da un PersistentVolume (PV) e un PersistentVolumeClaim (PVC). Il Service è di tipo ClusterIP sulla porta 5432.

TODO: Inserire qui uno screenshot dell'output di `kubectl get pods` con tutti i pod in stato Running (barebone).

I limiti principali di questa configurazione sono:

- Nessuna ridondanza: un guasto a qualsiasi Pod causa un'interruzione del servizio.
- Nessun Ingress Controller: l'accesso esterno alle API avviene esclusivamente tramite `kubectl port-forward` diretto verso ogni singolo servizio.
- Nessun load-balancing applicativo: non esiste alcun meccanismo per distribuire il carico tra istanze multiple in caso di picchi di traffico.



2.2 Infrastruttura con Ridondanza e Ingress

La seconda fase evolutiva dell'infrastruttura introduce due miglioramenti strutturali fondamentali: il **Kubernetes Ingress** e le **repliche multiple** per i servizi stateless critici.

Analisi della criticità e scelta delle repliche

La suddivisione logica del sistema in quattro layer guida la scelta delle repliche:

- **Frontend (nginx):** Stateless. 2 repliche per garantire disponibilità durante aggiornamenti o guasti di un Pod.
- **Logica di Business (auth, inventory, order):** Completamente stateless (lo stato risiede nel database). 3 repliche ciascuno, poiché questi servizi sono i punti di maggior stress durante accessi concorrenti: autenticazione simultanea di più utenti, letture/scritture sull'inventario, elaborazione parallela di ordini.
- **Database (postgres):** Rimane a 1 replica. La scalabilità orizzontale di un RDBMS relazionale richiede soluzioni architetturali dedicate (es. pgBouncer per il connection pooling, read-replica per le query di lettura, o servizi gestiti come Amazon RDS in produzione).

Il Service ClusterIP di Kubernetes funge automaticamente da **load-balancer interno**, distribuendo le richieste in modalità round-robin tra tutti i Pod attivi dello stesso Deployment.

Tutti i container dei servizi Flask sono inoltre dotati di `resources` (requests e limits) per evitare il problema del *noisy neighbor*: un Pod che consuma risorse eccessive non può degradare le prestazioni degli altri in esecuzione sullo stesso nodo fisico.

Kubernetes Ingress Controller

L'Ingress è un oggetto Kubernetes che agisce come **reverse proxy** HTTP a livello di cluster. Con un unico indirizzo IP esposto al pubblico, l'Ingress Controller (basato su NGINX) instrada le richieste verso il servizio backend corretto in base al prefisso del percorso URL, eliminando la necessità di port-forward multipli.

Le rotte implementate rispecchiano la suddivisione logica dell'API:

- `/` → `frontend-service` — serve le interfacce HTML statiche (cliente e amministratore).



- `/api/auth/` → `auth-service:5001` — endpoint di autenticazione, registrazione e gestione utenti.
- `/api/inventory/` → `inventory-service:5002` — endpoint catalogo, articoli e giacenze.
- `/api/orders/` → `order-service:5003` — endpoint ordini, storico e logistica.

TODO: Inserire qui un diagramma del flusso di rete: Client → Ingress → ClusterIP Service → Pod replica.

TODO: Inserire qui lo screenshot dell'output di `kubectl get ingress` e `kubectl get pods` con le repliche attive.

2.3 Guida all'Avvio

Fase 1 – Avvio Barebone

```
1 # Build immagini Docker (dalla root del progetto)
2 docker build -t cpiac_frontend:v5 -f frontend/Dockerfile
  frontend/
3 docker build -t cpiac_auth:v5      -f microservices/auth_service
  /Dockerfile .
4 docker build -t cpiac_inventory:v5 -f microservices/
  inventory_service/Dockerfile .
5 docker build -t cpiac_order:v5     -f microservices/
  order_service/Dockerfile .
6
7 # Deploy di tutte le risorse Kubernetes
8 kubectl apply -f k8s/
9
10 # Port-forward per accesso locale (alternativa all'Ingress)
11 kubectl port-forward svc/auth-service      5001:5001 &
12 kubectl port-forward svc/inventory-service 5002:5002 &
13 kubectl port-forward svc/order-service    5003:5003 &
14
15 kubectl get pods # Verifica stato
```

Listing 2.1: Avvio infrastruttura Barebone

Fase 2 – Avvio con Ridondanza e Ingress

```
1 # (Una-tantum) Installazione NGINX Ingress Controller
2 # In Minikube:
```



```
3 minikube addons enable ingress
4
5 # In un cluster generico (kubeadm, kind, cloud):
6 kubectl apply -f https://raw.githubusercontent.com/kubernetes/
  ingress-nginx/\
7 controller-v1.10.0/deploy/static/provider/cloud/deploy.yaml
8
9 # Deploy di tutte le risorse (i deployment aggiornati hanno gia
  ' le repliche)
10 kubectl apply -f k8s/
11
12 # Verifica repliche e stato Ingress
13 kubectl get pods
14 kubectl get ingress
15
16 # (Solo Minikube) Accesso via browser
17 minikube ip # Usa l'IP ottenuto per navigare su http://<
  minikube-ip>/
```

Listing 2.2: Avvio infrastruttura con repliche e Ingress

Per azzerare e ricreare l'intera infrastruttura da zero (ad esempio dopo una modifica strutturale al database):

```
1 kubectl delete -f k8s/
2 sleep 5
3 kubectl apply -f k8s/
```

Listing 2.3: Reset completo del cluster



Capitolo 3

Deployment in AWS

3.1 Deployment tramite Infrastructure-as-Code (Terraform)

Al fine di automatizzare e rendere riproducibile l'infrastruttura Cloud, è stato creato uno script Terraform per effettuare il provisioning delle risorse su Amazon Web Services (AWS). Questa metodologia, nota come *Infrastructure-as-Code* (IaC), garantisce che la configurazione del cloud sia scritta in file dichiarativi, riducendo l'errore umano.

I file Terraform implementati (`main.tf`, `variables.tf`, `outputs.tf`) descrivono la seguente architettura:

- **VPC e Subnet:** Creazione di una Virtual Private Cloud (VPC) con subnet pubbliche per bilanciatori e risorse accessibili, e subnet private per i nodi Kubernetes.
- **Internet Gateway e Route Tables:** Per consentire il routing verso l'esterno.
- **Security Group:** Regole firewall stateful per aprire le porte in ingresso al traffico HTTP/HTTPS e porte interne per la comunicazione intra-nodo.
- **Application Load Balancer (ALB):** Per distribuire il traffico degli utenti verso i worker node.
- **Istanze EC2:** Lancio di una singola macchina virtuale (nodo unificato) di classe `t3.medium` basata su Amazon Machine Image (AMI) Ubuntu per limitare i costi di esecuzione. Lo script `user_data.sh` automatizza l'installazione di Docker e delle componenti di Kubernetes (K3s) all'avvio.
- **AWS Budgets:** Per evitare costi imprevisti, è configurato un *Monthly Budget* con limite di spesa pari a 100 USD mensili, che invia alert via email se i costi superano la soglia dell'80%. L'utilizzo di un singolo nodo garantisce il rispetto di tale budget pur mantenendo l'Application Load Balancer.



TODO: Inserire qui uno schema grafico dell'Architettura AWS Standard (VPC, Security Group, EC2, ALB).

I passi per eseguire lo script sono i seguenti:

1. Inizializzazione dell'ambiente con `terraform init`.
2. Verifica del piano di esecuzione con `terraform plan`.
3. Creazione reale delle risorse con `terraform apply`.

TODO: Inserire qui uno screenshot dell'output del terminale dopo aver eseguito `terraform apply` con successo.

3.2 Deployment tramite AWS Management Console (GUI)

In alternativa, è possibile creare manualmente la stessa infrastruttura descritta al paragrafo precedente direttamente tramite il portale web di AWS. Questo approccio è utile a scopo esplorativo e didattico.

Di seguito i passaggi principali:

1. **Creazione VPC:** Dalla sezione VPC, utilizzare il wizard "VPC and more" per creare la rete, specificando il CIDR block e includendo un Internet Gateway.
2. **Configurazione Subnet e Route Table:** Separare logicamente le reti pubbliche e private.
3. **Lancio Istanze EC2:** Dalla dashboard EC2, cliccare su "Launch Instance". Selezionare un'AMI Ubuntu, scegliere una classe di istanza (es. `t3.medium`), associare una coppia di chiavi SSH e collocare l'istanza nella subnet corretta associandole un Security Group che apra la porta 22 (SSH) e la 80 (HTTP).
4. **Creazione Load Balancer:** Dalla sezione "Load Balancing", creare un ALB, associarlo alle subnet pubbliche e creare un *Target Group* che punti alle istanze EC2 su cui gira Kubernetes.
5. **Impostazione AWS Budget:** Dalla sezione "Billing", accedere a "Budgets" e creare un budget di costo mensile pari a 100\$, specificando l'indirizzo email per gli alert al superamento dell'80%.

TODO: Inserire qui alcuni screenshot della console AWS (es. lista delle istanze EC2 in esecuzione e la dashboard del Load Balancer).



Capitolo 4

Proposte di Integrazione e Sviluppo Futuro

Questo capitolo presenta le proposte per l'evoluzione dell'architettura e per l'esecuzione di test prestazionali.

4.1 Panoramiche sull'utilizzo in locale (On-Premise / Edge Computing)

L'architettura a microservizi basata su K3s (una versione leggera di Kubernetes) è perfetta per essere eseguita anche al di fuori del Cloud, sfruttando l'hardware locale.

Scenario A: Cluster On-Premise con PC Fissi e Portatili

Invece di usare macchine virtuali su AWS, è possibile creare un **cluster fisico locale**.

- **Come funziona:** Un PC desktop mediamente potente viene designato come **Master Node** (Control Plane). Si collegano poi alla stessa rete LAN 2 o 3 portatili che fungeranno da **Worker Node**.
- **Vantaggi didattici e operativi:** Questo scenario simula perfettamente un vero Data Center aziendale. Permette di dimostrare l'**High Availability (HA)** spegnendo fisicamente un portatile (simulando un guasto hardware) e osservando Kubernetes che riavvia i Pod (i microservizi) sugli altri nodi superstiti.

Scenario B: Utilizzo di microcontrollori (ESP32 / ESP8266) nel sistema

Schede come l'ESP32 o l'ESP8266 **non hanno i requisiti minimi** per eseguire Docker o Kubernetes, avendo risorse di memoria estremamente limitate. Tuttavia, si integrano perfettamente nel sistema agendo da **IoT Edge Devices** (dispositivi periferici).



- **Casi d'uso pratici per Scalamarket:**

1. **Lettore di codici a barre Smart (Warehouse):** Un ESP32 collegato a un modulo scanner. Quando l'operatore di filiale scansiona una merce, l'ESP32 invia una richiesta HTTP POST via Wi-Fi direttamente a `/api/inventory/` per aggiornare il database delle scorte in tempo reale.
2. **Monitoraggio Flotta (Veicoli):** Un ESP8266 installato sui mezzi logistici, dotato di modulo GPS, esegue chiamate REST periodiche verso l'`order-service` per aggiornare la posizione del mezzo e lo stato della consegna.

4.2 Benchmark Prestazionali (Richieste/sec)

Per testare l'efficacia della scalabilità introdotta nella piattaforma, si propone l'utilizzo di strumenti gratuiti per load testing come **Locust** (basato su Python) o **Apache Bench (ab)**.

Test 1: Sfogliare il Catalogo (Read-Intensive)

- **Azione:** Effettuare migliaia di chiamate GET `/api/inventory/` per scaricare la lista dei prodotti.
- **Obiettivo:** Misurare le **RPS (Richieste per Secondo)** massime che il sistema riesce a servire restando sotto i 200 millisecondi di latenza.

Test 2: Inserimento Ordini (Write-Intensive e Logica)

- **Azione:** Effettuare chiamate POST `/api/orders/` con payload JSON validi, simulando centinaia di clienti che effettuano il checkout contemporaneamente.
- **Obiettivo:** Valutare come si comporta il sistema sotto stress in presenza di transazioni sul database e logica condizionale di allocazione. Confrontando la fase Barebone (1 Pod) con la fase Scalabile (3 Pod), si evidenzierà il reale impatto dello scaling orizzontale.

Test 3: Spike Test (Picco Improvviso)

- **Azione:** Aumentare bruscamente gli utenti concorrenti (da 10 a 1000 in pochi secondi) su tutte le API.



Scalamarket: Using API, Kubernetes and AWS for developing a Microservice-based E-Commerce Platform

- **Obiettivo:** Verificare se l'Ingress restituisce errori 502/503/504 per proteggere i Pod da crolli (OOM).



Capitolo 5

Documentazione Endpoints API

Questo documento presenta tutti gli endpoint implementati nell'applicativo Scalamarket. Il sistema è basato su un'architettura a microservizi, composta da: Auth Service (Porta 5001), Inventory Service (Porta 5002) e Order Service (Porta 5003). Di seguito, le tabelle sono suddivise in base ai file di routing (es. `client.py` e `admin.py`) che implementano i vari endpoint.

Auth Service

/client/register			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Registra un nuovo utente cliente nel sistema.	{username, password}	201 (Created) 400 (Bad Request)
/client/login			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Autentica un cliente e ne restituisce le informazioni.	{username, password}	200 (OK) 401 (Unauthorized)

Tabella 5.1: Auth Service - `routes/client.py`



/admin/register			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Registra un nuovo amministratore o gestore filiale.	{username, password, role, location_id}	201 (Created) 400 (Bad Request)
/admin/login			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Autentica un amministratore e ne restituisce i dati.	{username, password}	200 (OK) 401 (Unauthorized)
/admin/users			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Restituisce la lista di tutti gli utenti registrati.	Nessuno	200 (OK)
/admin/users/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
DELETE	Elimina definitivamente l'utente specificato dall'ID.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)

Tabella 5.2: Auth Service - routes/admin.py



Inventory Service

/client/catalog			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Ottiene il catalogo completo degli articoli disponibili per i clienti.	Nessuno	200 (OK)

Tabella 5.3: Inventory Service - routes/client.py



/admin/article			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Aggiunge un nuovo articolo all'inventario con relative giacenze.	{name, description, price, quantities}	201 (Created) 400 (Bad Request)
/admin/inventory			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Visualizza lo stato globale di inventario e giacenze.	Nessuno	200 (OK)
/admin/inventory			
	Descrizione	Parametri Richiesti	Codici HTTP
PUT	Aggiorna la quantità di giacenza di un articolo in una specifica filiale.	{article_id, location_id, quantity}	200 (OK) 400 (Bad Request)
/admin/article/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
DELETE	Rimuove definitivamente un articolo dal catalogo e tutte le sue giacenze.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)
/admin/inventory/<article_id>/<location_id>			
	Descrizione	Parametri Richiesti	Codici HTTP
DELETE	Rimuove una specifica voce di inventario per un articolo in una filiale.	Nessuno (nell'URL)	200 (OK) 404 (Not Found)
/admin/internal/allocate			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Effettua l'allocazione logistica prelevando gli articoli dalle filiali.	{order_id, items}	200 (OK) 400 (Bad Request)

Tabella 5.4: Inventory Service - routes/admin.py



Order Service

/client/order			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Crea e piazza un nuovo ordine a nome dell'utente specificato.	{user_id, items}	201 (Created) 400 (Bad Request)
/client/order/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Traccia lo stato di uno specifico ordine in base all'ID fornito.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)
/client/order/user/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Ottiene lo storico di tutti gli ordini completati e falliti dell'utente.	Nessuno (id nell'URL)	200 (OK)

Tabella 5.5: Order Service - routes/client.py

/admin/orders			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Restituisce tutti gli ordini registrati nel sistema.	Nessuno	200 (OK)
/admin/logistics			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Visualizza l'uso corrente e totale dei veicoli logistici per le filiali.	Nessuno	200 (OK)
/admin/logistics			
	Descrizione	Parametri Richiesti	Codici HTTP
PUT	Aggiorna i veicoli logistici totali di una determinata filiale.	{location_id, total_vehicles}	200 (OK) 404 (Not Found)

Tabella 5.6: Order Service - routes/admin.py